

Notice des scripts

Étude de la diffusion de contenu sur les réseaux sociaux

Léore Bensabath Ilyas Hanine

Cette notice décrit les scripts Python accompagnant le projet. Ils sont regroupés selon les trois approches du rapport : modélisation par graphes, modèle compartimental SIR, et processus ponctuels de Hawkes. Pour chaque script sont indiqués son objet global puis le rôle de chaque fonction.

1 Utilitaires de génération de graphes

`generer_graphe.py` Génère un graphe orienté pondéré aléatoire servant de support de réseau.

- `generer_graphe(N)` : construit la liste d'adjacence de N nœuds ; pour chaque nœud, tire un nombre de voisins uniforme et leur affecte des poids aléatoires dans $[0, 1]$, sans répétition.

`Poisson.py` Court utilitaire de tirage (fichier de test).

- `Poisson(l)` : renvoie un entier dérivé d'un tirage exponentiel de paramètre l , comme approximation rudimentaire d'un temps inter-arrivées.

2 Approche par graphes

`Cascade.py` Modèle de cascade indépendante : un nœud actif tente *une seule fois* d'infecter chaque voisin, avec une probabilité égale au poids de l'arc.

- `generer_grapheProfil(N)` : génère le graphe avec un nombre de voisins poissonnien (moyenne $N/3$) et des poids réduits ($\propto 1/N$) pour des probabilités réalistes.
- `generer_infecte(N, graphe)` : tire N nœuds initialement infectés, codés en tuples (nœud, parent = -1, temps = 0).
- `parcours_largeur(Profil, I)` : réalise la diffusion par parcours en largeur ; pour chaque voisin d'un infecté, un tirage uniforme comparé au poids décide de la transmission ; renvoie les nœuds atteints et leur temps d'infection.
- `comptage_chemin(Profil, Infecte)` : à partir du parcours, reconstruit la courbe du nombre cumulé d'infectés au cours du temps, jusqu'à stabilisation.

`seuil.py` Modèle à seuil linéaire, déterministe : un nœud s'active dès que la somme des poids de ses voisins actifs dépasse son seuil propre.

- `generer_grapheSeuil(N)` : génère le graphe pondéré et attribue à chaque nœud un seuil égal à la somme des poids d'un sous-ensemble aléatoire de ses voisins.
- `generer_infecte(N, graphe)` : initialise un vecteur booléen d'infectés en activant N nœuds au hasard.
- `modele_seuil(graphe, tmax, NombreInfecte)` : à chaque pas de temps, active les nœuds dont le cumul des poids des voisins actifs atteint le seuil ; renvoie pour chaque nœud son état final et son temps d'activation.

3 Modèle SIR

`modele_SIR.py` Modèle SIR déterministe, résolu par intégration numérique des équations différentielles.

- `SIR(Q, t, b, g, N)` : définit le système d'équations différentielles $(\dot{Q}_S, \dot{Q}_I, \dot{Q}_R)$, intégré par `scipy.integrate.odeint`.

`SIR_WIKI2.py` Simulation SIR stochastique en temps discret (schéma chaîne binomiale). *Implémentation de référence, adaptée d'un script externe*. Sans fonction : une boucle tire, à chaque pas, les nouvelles infections (probabilité $\beta I/N$ par susceptible) et les guérisons (probabilité γ), puis enregistre S, I, R .

`SIR_wiki.py` Simulation SIR stochastique en temps continu (algorithme de Gillespie, avec démographie μ). *Implémentation de référence, adaptée d'un script externe*.

- `stoc_eqs(INP, ts)` : calcule les six taux d'événements (infection, guérison, naissances et morts), applique l'événement tiré et renvoie l'état mis à jour ainsi que le pas de temps.
- `Stoch_Iteration(INPUT)` : itère `stoc_eqs` jusqu'à la durée maximale et accumule les trajectoires T, S, I, R .

4 Approche stochastique : processus de Hawkes

`HP_thinning.py` Génère un processus de Hawkes par amincissement (*thinning*) et trace son intensité et son processus de comptage.

- `lambda_etoile(l, a, b, H, t)` : évalue l'intensité conditionnelle $\lambda^*(t) = l + \sum_{\tau < t} a e^{-b(t-\tau)}$, avec un facteur de saturation $(1 - r/200)$ limitant l'auto-excitation.
- `HP_by_thinning(l, a, b, T)` : algorithme d'Ogata ; propose des temps candidats et les accepte ou rejette par amincissement, renvoyant la liste des temps d'événements.
- `Count(H)` : renvoie le processus de comptage $(0, 1, 2, \dots)$ associé aux temps H .

`hawkes_process.py` Seconde implémentation du Hawkes par thinning, conservant et affichant à la fois les points retenus et rejetés.

- `intensite(lambda0, alpha, beta, P, t)` : intensité conditionnelle calculée à partir des temps déjà retenus P .
- `escalier(x, y)` : trace une fonction en escalier pour visualiser le comptage.
- `hawkesbythinning(lambda0, alpha, beta, T)` : génère le processus, stocke séparément les points acceptés (P) et rejetés (Q) avec leurs intensités, et les visualise.

`SIR_hawkes.py` Couple le modèle SIR à deux processus de Hawkes (infectés et retirés), construits récursivement. *À corriger : une parenthèse manquante dans `lambda_etoileHi` empêche l'exécution en l'état.*

- `lambda_etoileHr(l, a, b, H, t)` : intensité du processus des retirés (avec saturation $1 - r/200$).
- `lambda_etoileHi(l, a, b, H, Hr, t)` : intensité du processus des infectés, ne comptant que les infectés non encore retirés.
- `HP_by_thinningHi(...)` et `HP_by_thinningHr(...)` : tirent par amincissement le prochain temps d'infection, respectivement de retrait, à partir de l'instant courant.
- `SIR(Qr0, Qs0, Qi0)` : avance le système en retenant le plus petit temps des deux processus, met à jour les comptages et les populations Q_S, Q_I, Q_R , puis trace les trajectoires.

`simulations3.py` Construit le SIR à partir d'un processus de Poisson non homogène par inversion du temps, en alternative au thinning.

- `Lambda(tupper)` : intègre l'intensité $\lambda(x) = \text{tupper} \cdot x$ sur $[0, \text{tupper}]$ (compensateur).
- `lambda_inv(s, t_max)` : inverse numériquement le compensateur, renvoyant le plus petit temps tel que $\Lambda(t) \geq s$.
- `poisson_process(tmax)` : génère les temps d'un processus de Poisson par la méthode d'inversion (*time-rescaling*).
- `fonction_principale(QI0, QSO, QRO)` : construit récursivement les trajectoires S, I, R en choisissant à chaque étape le plus petit incrément (infection ou retrait) ; la fonction interne `actualiserf` met à jour les compensateurs f_I, f_R ; les courbes sont tracées en temps logarithmique.

`SIR_poisson.py` Ébauche d'un SIR piloté par processus de comptage. *Script incomplet : paramètres non renseignés et corps de la boucle SIR vide, donc non exécutable en l'état.* Il réutilise `lambda_etoile`, `HP_by_thinning` et `Count` (cf. ci-dessus) et amorce une fonction SIR reliant les compensateurs f_I et f_R .